



MODUL PERKULIAHAN **MATEMATIKA** **4**

Kode Mata kuliah W132500014

Modul

Metode Jumlahan Pecahan Parsial

Abstrak

Pertemuan ini membahas Metode Jumlahan Pecahan Parsial (Partial Fraction Decomposition/PFD): teknik

Sub - CPMK

Sub-CPMK 4.2 – Mampu menentukan fungsi tangga satuan, metode jumlahan parsial dan teorema konvolusi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Engineering-Mathematics/Modul/Modul-13.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

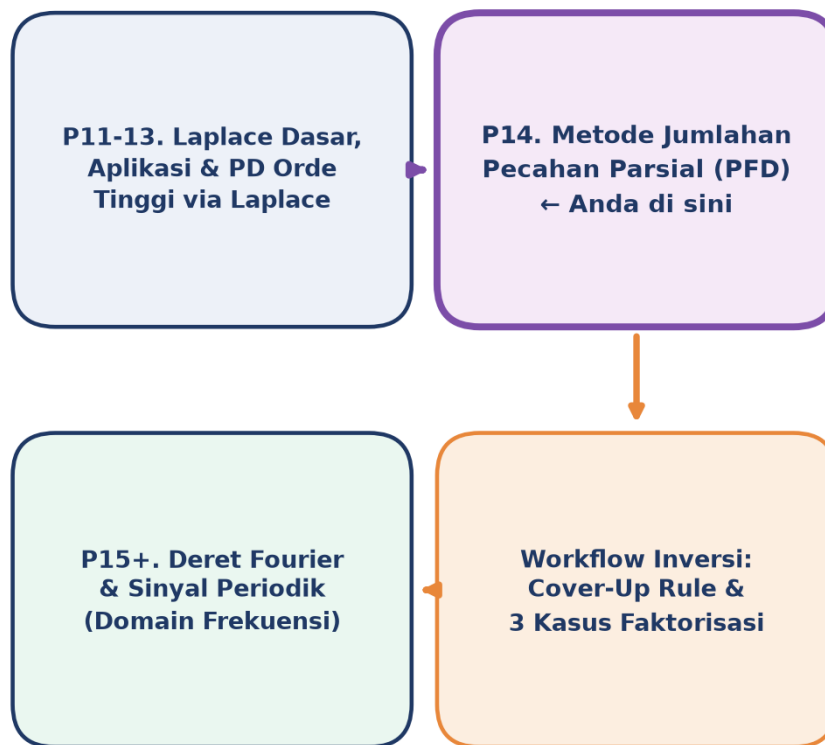
PENDAHULUAN

Pertemuan 11 sampai 13 membangun toolkit Transformasi Laplace secara bertahap: definisi dan tabel dasar, aplikasi lanjut (Heaviside, Dirac, transfer function, konvolusi), lalu fungsi tangga satuan sebagai representasi sinyal diskontinu. Setiap kali Laplace diterapkan pada Persamaan Diferensial, hasil akhirnya hampir selalu berupa fungsi rasional $F(s) = P(s)/Q(s)$ di domain s , rasio dua polinomial. Masalahnya, tabel invers Laplace hanya memuat bentuk-bentuk sederhana: $1/(s-a)$, $1/(s-a)^2$, $\omega/(s^2+\omega^2)$, dan sejenisnya. $F(s)$ hasil aljabar PD orde tinggi jarang langsung cocok dengan salah satu baris tabel tersebut. Di sinilah Metode Jumlahan Pecahan Parsial (Partial Fraction Decomposition/PFD) mengambil peran sentral: teknik aljabar untuk memecah $F(s)$ yang rumit menjadi jumlah pecahan-pecahan sederhana, masing-masing langsung cocok dengan satu baris tabel invers Laplace.

Fokus Pertemuan 14: Pertemuan 14 ini menyajikan PFD secara sistematis lewat tiga kasus faktorisasi penyebut $Q(s)$: faktor linear berbeda (akar riil tunggal), faktor linear berulang (akar riil dengan multiplicity lebih dari satu), dan faktor kuadratik tak tereduksi (akar kompleks konjugat, diskriminan negatif). Tiap kasus memiliki template dekomposisi dan teknik pencarian koefisien tersendiri. Untuk kasus faktor linear non-berulang, tersedia shortcut elegan bernama cover-up rule (dinamai dari insinyur-fisikawan Inggris Oliver Heaviside), yang memangkas waktu pengerjaan drastis dibanding menyamakan koefisien polinomial penuh. Modul ditutup dengan workflow inversi Laplace lengkap end-to-end dan lima aplikasi PFD di luar solve PD: integral fungsi rasional, dekomposisi modal di control system, filter design di signal processing, hingga polynomial manipulation di numerical analysis.

Perspektif Historis dan Pedagogis: Metode operasional untuk menyelesaikan persamaan diferensial lewat transformasi aljabar, cikal bakal Transformasi Laplace modern, mula-mula dikembangkan Oliver Heaviside pada akhir abad ke-19 sebagai alat praktis insinyur telegraf, jauh sebelum landasan matematisnya diformalkan secara rigoros oleh matematikawan generasi berikutnya. Cover-up rule yang dipelajari pada pertemuan ini adalah warisan langsung dari pendekatan operasional Heaviside tersebut, sering disebut

Heaviside method atau Heaviside expansion theorem dalam literatur klasik. Menariknya, meski usianya lebih dari satu abad, teknik ini tetap menjadi materi wajib di hampir semua kurikulum matematika teknik modern karena kesederhanaan dan efisiensinya belum tertandingi untuk kasus faktor linear non-berulang. Riset pedagogis kontemporer bahkan masih meneliti cara mengajarkan PFD secara lebih efektif kepada mahasiswa; studi terbaru menemukan bahwa metode alternatif berbasis substitusi sistematis dapat mengurangi kesalahan komputasi mahasiswa dibanding pendekatan tradisional menyamakan koefisien penuh (Jong & Kuan, 2020). Pola yang berulang di sini konsisten dengan pertemuan-pertemuan sebelumnya: kebutuhan praktis rekayasa mendahului kerapihan teori, namun keduanya akhirnya bertemu menjadi fondasi kokoh yang diajarkan turun-temurun.



Gambar 1. Posisi Pertemuan 14 (Metode Jumlahan Pecahan Parsial) dalam alur mata kuliah Matematika 4, menjembatani toolkit Laplace dari Pertemuan 11-13 menuju Deret Fourier di Pertemuan 15.

Struktur Modul: Gambar 1 memperlihatkan bahwa Pertemuan 14 adalah simpul penting: seluruh hasil aljabar Laplace dari Pertemuan 11-13 (PD orde tinggi, Heaviside, Dirac, transfer function, konvolusi) pada akhirnya bermuara ke fungsi rasional $F(s)$ yang harus di-invers, dan PFD adalah jembatan wajib menuju $f(t)$. Setelah PFD dikuasai, Pertemuan 15 akan beralih ke Deret Fourier, perspektif komplementer untuk sinyal periodik di domain frekuensi yang tidak melalui rute Laplace-PFD.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 4.2:** Mampu menentukan fungsi tangga satuan, metode jumlahan parsial dan teorema konvolusi; pada Pertemuan 14 fokus khusus pada penguasaan tiga kasus dekomposisi pecahan parsial dan penerapannya pada inversi Laplace.
- Setelah pertemuan ini mahasiswa dapat: mengklasifikasikan faktor penyebut $Q(s)$ menjadi tiga kasus (linear berbeda, linear berulang, kuadratik tak tereduksi); menulis template dekomposisi pecahan parsial yang tepat untuk tiap kasus; menghitung koefisien lewat cover-up rule (kasus linear non-berulang) dan menyamakan koefisien polinomial (kasus berulang dan kuadratik); menjalankan workflow inversi Laplace lengkap dari $F(s)$ hingga $f(t)$; serta memverifikasi hasil secara numerik dan lewat SymPy `sp.apart()`.

BENTUK UMUM PFD DAN FAKTORISASI $Q(S)$

Syarat Utama: $\deg(P) < \deg(Q)$. Sebelum dekomposisi dapat dilakukan, $F(s) = P(s)/Q(s)$ harus memenuhi satu syarat ketat: $\deg(P)$ kurang dari $\deg(Q)$, disebut fungsi rasional proper. Jika $\deg(P)$ lebih besar atau sama dengan $\deg(Q)$ (improper), langkah pertama wajib adalah polynomial long division: $P(s)/Q(s) = K(s) + R(s)/Q(s)$, dengan $K(s)$ polinomial (mudah di-invers, melibatkan delta Dirac dan turunannya) dan $R(s)/Q(s)$ sudah proper, siap untuk PFD. Contoh improper: $F(s) = (s^3+1)/(s^2-1)$; long division memberi $F(s) = s + (s+1)/(s^2-1) = s + 1/(s-1)$ setelah faktor $(s+1)$ saling tereduksi.

$$P(s)/Q(s) = \sum A_i/(s-a_i) + \sum B_j/(s-b_j)^k + \sum (Cs+D)/(s^2+ps+q), \text{ syarat: } \deg(P) < \deg(Q)$$

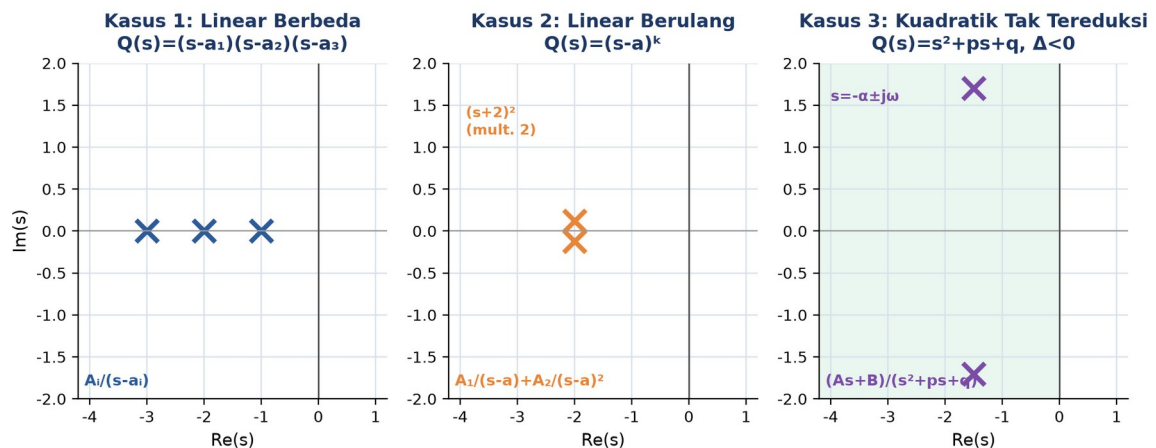
Faktorisasi $Q(s)$ atas Bilangan Riil. Setelah syarat proper terpenuhi, langkah berikutnya adalah memfaktorkan $Q(s)$ atas bilangan riil. Teorema aljabar menjamin bahwa polinomial berkoefisien riil apapun selalu dapat difaktorkan atas riil menjadi kombinasi dua jenis faktor dasar: faktor linear $(s-a)$ dari akar riil, dan faktor kuadratik tak tereduksi (s^2+ps+q) dengan diskriminan negatif dari pasangan akar kompleks konjugat. Kedua jenis faktor ini dapat muncul berulang (multiplicity lebih dari satu). Kombinasi keduanya menghasilkan tiga kasus dekomposisi PFD yang menjadi inti pembahasan pertemuan ini.

Contoh, Faktorisasi Campuran. Tentukan template PFD dari $F(s) = (s+1)/[s^2 \cdot (s^2+4)]$. Faktorkan penyebut: $Q(s) = s^2 \cdot (s^2+4)$. Klasifikasi faktor: s^2 adalah faktor linear berulang (akar $s=0$ dengan multiplicity 2), sedangkan s^2+4 adalah faktor kuadratik tak tereduksi (diskriminan $0-16 = -16$ lebih kecil dari nol, akar kompleks $s = \pm 2j$). Template gabungan: $F(s) = A/s + B/s^2 + (Cs+D)/(s^2+4)$. Verifikasi syarat: $\deg(\text{pembilang})=1$, $\deg(\text{penyebut})=4$, proper.

Tabel 1 merangkum tiga kasus dekomposisi PFD berdasarkan jenis faktor $Q(s)$, lengkap dengan template dan cara menentukan koefisiennya. Ketiga kasus ini menjadi kerangka utama Bagian selanjutnya modul ini.

Tabel 1. Tiga kasus dekomposisi pecahan parsial berdasarkan jenis faktor penyebut $Q(s)$.

Kasus	Bentuk Faktor $Q(s)$	Template PFD	Cara Hitung Koefisien
1. Linear berbeda	$(s-a_1)(s-a_2)\dots(s-a_n)$	$A_1/(s-a_1)+\dots+A_n/(s-a_n)$	Cover-up rule (langsung)
2. Linear berulang	$(s-a)^k, k \geq 2$	$A_1/(s-a)+A_2/(s-a)^2+\dots+A_k/(s-a)^k$	Cover-up utk A_k , turunan/samakan koefisien utk sisanya
3. Kuadratik tak tereduksi	(s^2+ps+q) , diskriminan < 0	$(As+B)/(s^2+ps+q)$	Samakan koefisien polinomial



Gambar 2. Visualisasi tiga kasus faktorisasi $Q(s)$ di s -plane: pole real berbeda (kiri), pole real berulang multiplicity 2 (tengah), dan pole kompleks konjugat dari faktor kuadratik tak tereduksi (kanan).

Membaca Tiga Panel di Atas: Gambar 2 menegaskan bahwa lokasi akar $Q(s)$ di s -plane secara langsung menentukan bentuk template PFD: akar real tunggal memberi satu term $A/(s-a)$, akar real berulang memberi rangkaian term berpangkat naik, dan pasangan akar kompleks konjugat digabung menjadi satu faktor kuadratik dengan numerator linier $(As+B)$. Klasifikasi ini wajib dilakukan sebelum menulis template, sebab template yang salah membuat seluruh perhitungan koefisien berikutnya tidak valid.

Tiga Pitfall Umum Sebelum PFD: Pitfall paling umum pada tahap ini adalah lupa mengecek syarat proper sebelum mulai PFD, sehingga hasil dekomposisi salah sejak awal. Pitfall kedua adalah salah mengklasifikasikan kuadratik: faktor (s^2+ps+q) dengan diskriminan lebih besar atau sama dengan nol sebenarnya bukan kasus 3, melainkan produk dua faktor linear yang harus difaktorkan lebih lanjut (masuk kasus 1 atau 2). Pitfall ketiga adalah salah menghitung multiplicity akar berulang; SymPy `sp.factor()` dan `sp.roots()`

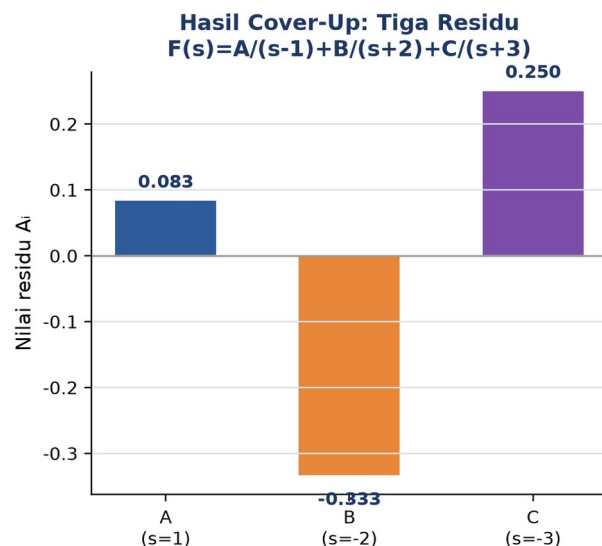
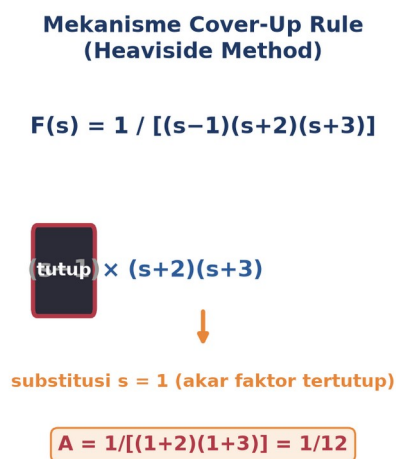
sangat membantu memverifikasi faktorisasi manual sebelum melangkah ke penentuan koefisien.

KASUS 1 DAN KASUS 2: FAKTOR LINEAR VIA COVER-UP RULE

Kasus 1, Faktor Linear Berbeda. Kasus paling sering muncul di soal PD adalah faktor linear berbeda: $Q(s) = (s-a_1)(s-a_2)\dots(s-a_n)$ dengan seluruh akar riil dan berbeda satu sama lain. Template PFD-nya deterministik: $F(s) = A_1/(s-a_1) + A_2/(s-a_2) + \dots + A_n/(s-a_n)$, tanpa ambiguitas. Untuk kasus ini tersedia shortcut sangat cepat bernama cover-up rule: tutup faktor $(s-a_i)$ di penyebut $F(s)$, lalu evaluasi sisa ekspresi pada $s = a_i$. Hasilnya langsung sama dengan A_i , tanpa perlu menyusun dan menyelesaikan sistem persamaan linier.

$$A_i = \lim_{s \rightarrow a_i} (s-a_i)F(s) = P(a_i) / \text{produk}(j \neq i)(a_i - a_j); \text{"tutup (s-a_i), substitusi s=a_i ke sisa"}$$

Mengapa Cover-Up Bekerja: Derivasi cover-up rule cukup singkat. Dari $F(s) = A_1/(s-a_1) + A_2/(s-a_2) + \dots$, kalikan kedua ruas dengan $(s-a_1)$: $(s-a_1)F(s) = A_1 + (s-a_1)[A_2/(s-a_2) + \dots]$. Ambil limit s menuju a_1 : suku kedua menuju nol karena faktor $(s-a_1)$ pada pembilangnya, sehingga tersisa $A_1 = \lim_{s \rightarrow a_1} (s-a_1)F(s)$. Hasil ini identik dengan rumus residu $A_1 = P(a_1)/Q'(a_1)$ di teori fungsi kompleks, sebab A_i adalah residu $F(s)$ pada pole sederhana a_i .



Gambar 3. Mekanisme cover-up rule pada $F(s) = 1/[(s-1)(s+2)(s+3)]$: faktor $(s-1)$ ditutup lalu $s=1$ disubstitusi ke sisa penyebut (kiri), menghasilkan tiga residu $A=1/12$, $B=-1/3$, $C=1/4$ (kanan).

Membaca Ilustrasi Cover-Up: Gambar 3 mengilustrasikan langkah cover-up secara konkret: setelah faktor $(s-1)$ ditutup, tersisa $1/[(s+2)(s+3)]$; substitusi $s=1$ memberi $1/[(1+2)(1+3)] = 1/12 = A$. Prosedur identik diulang untuk faktor $(s+2)$ pada $s=-2$ dan $(s+3)$ pada $s=-3$, menghasilkan $B=-1/3$ dan $C=1/4$. Ketiga residu ini langsung menjadi koefisien template

$F(s) = A/(s-1) + B/(s+2) + C/(s+3)$, siap di-invers via tabel menjadi $f(t) = (1/12)e^t - (1/3)e^{-2t} + (1/4)e^{-3t}$.

Contoh 1, Dua Faktor Linear Berbeda. Tentukan invers Laplace dari $F(s) = (2s+3)/(s^2-5s+6)$ memakai cover-up rule. Faktorkan penyebut: $s^2-5s+6 = (s-2)(s-3)$, akar $s=2$ dan $s=3$. Template: $F(s) = A/(s-2) + B/(s-3)$. Cover-up A (tutup $s-2$, substitusi $s=2$): $A = (2*2+3)/(2-3) = 7/(-1) = -7$. Cover-up B (tutup $s-3$, substitusi $s=3$): $B = (2*3+3)/(3-2) = 9/1 = 9$.

$$\text{PFD: } F(s) = 9/(s-3) - 7/(s-2) \Rightarrow f(t) = 9e^{3t} - 7e^{2t}$$

Kasus 2, Faktor Linear Berulang. Kasus kedua muncul bila $Q(s)$ memiliki akar berulang $(s-a)^k$ dengan multiplicity k lebih besar atau sama dengan 2. Satu term $A/(s-a)$ tidak lagi cukup untuk menampung informasi struktural akar berulang tersebut; diperlukan k term dengan pangkat berbeda: $A_1/(s-a) + A_2/(s-a)^2 + \dots + A_k/(s-a)^k$. Cover-up rule sederhana hanya memberi koefisien pangkat tertinggi A_k secara langsung (kalikan $F(s)$ dengan $(s-a)^k$ penuh, lalu substitusi $s=a$); koefisien pangkat lebih rendah (A_1 sampai A_{k-1}) membutuhkan teknik turunan atau menyamakan koefisien polinomial secara manual.

$$A_k = (s-a)^k * F(s) |_{s=a}; \quad A_{k-j} = (1/j!) * d^j/ds^j [(s-a)^k * F(s)] |_{s=a}, \quad j=1,2,\dots,k-1$$

Contoh 2, Resonansi dan Multiplicity 4. Selesaikan $y'' - 4y' + 4y = 6t^2e^{2t} - 4e^{2t}$ dengan $y(0)=2$, $y'(0)=8$. Karakteristik: $s^2-4s+4 = (s-2)^2$, akar berulang $s=2$ (critically damped). Karena input juga mengandung e^{2t} , pole input bertabrakan dengan pole sistem, menghasilkan resonansi: multiplicity total $Y(s)$ naik menjadi 4. Hasil transformasi $Y(s) = (2s^3-8s^2+4s+14)/(s-2)^4$. Template PFD memerlukan empat term berpangkat 1 sampai 4: $Y(s) = A_1/(s-2) + A_2/(s-2)^2 + A_3/(s-2)^3 + A_4/(s-2)^4$. Cover-up pangkat tertinggi memberi $A_4=6$ (kalikan $(s-2)^4$ penuh, substitusi $s=2$); koefisien $A_3=-4$ diperoleh lewat satu kali turunan terhadap $[(s-2)^4 Y(s)]$ sebelum substitusi $s=2$.

$$L^{-1}\{1/(s-a)^n\} = t^{n-1}e^{at}/(n-1)!; \quad \text{contoh: } 1/(s-a) \rightarrow e^{at}, \quad 1/(s-a)^2 \rightarrow te^{at}, \\ 1/(s-a)^3 \rightarrow t^2e^{at}/2$$

Penyelesaian Lengkap Contoh 2: Menuntaskan Contoh 2 di atas: substitusi $u=s-2$ pada numerator memberi $2s^3-8s^2+4s+14 = 2u^3+4u^2-4u+6$, sehingga $Y(s) = 2/u + 4/u^2 - 4/u^3 + 6/u^4$, artinya $A_1=2$, $A_2=4$ (konsisten dengan $A_3=-4$ dan $A_4=6$ yang sudah dihitung lewat cover-up dan turunan). Inversi tiap term via tabel $L^{-1}\{1/(s-a)^n\}$ di atas memberi $y(t) = 2e^{2t} + 4te^{2t} + (-4/2!)t^2e^{2t} + (6/3!)t^3e^{2t}$, yang setelah faktor e^{2t} dikeluarkan menjadi bentuk polinomial tunggal dikali eksponensial. Verifikasi IVP: $y(0) = 2$ (sesuai syarat awal), dan turunan $y'(t) = 2e^{2t}(2+4t-2t^2+t^3) + e^{2t}(4-4t+3t^2)$ pada $t=0$ memberi $y'(0) = 2*2+4 = 8$, cocok persis dengan syarat awal $y'(0)=8$ pada soal. Kecocokan kedua syarat awal ini menjadi bukti bahwa keempat koefisien A_1 sampai A_4 sudah benar, sekaligus menegaskan bahwa teknik turunan berulang pada faktor

linear multiplicity tinggi, meski lebih panjang dibanding cover-up sederhana, tetap sistematis dan dapat diverifikasi langkah demi langkah.

Hasil: $y(t) = e^{(2t)} * (2 + 4t - 2t^2 + t^3)$, verifikasi $y(0)=2$ dan $y \text{ prime}(0)=8$ sesuai IVP

Tabel 2 merangkum rumus turunan lengkap untuk menghitung seluruh koefisien A_k -j pada faktor linear berulang, dilengkapi contoh konkret pada kasus multiplicity 2 dan 3 yang paling sering muncul di soal ujian.

Tabel 2. Rumus turunan untuk koefisien faktor linear berulang $(s-a)^k$, cover-up hanya berlaku untuk pangkat tertinggi.

Multiplicity k	Koefisien Pangkat Tertinggi A_k	Koefisien Lebih Rendah
k=2	$A_2 = (s-a)^2 F(s) _{s=a}$ (cover-up)	$A_1 = d/ds[(s-a)^2 F(s)] _{s=a}$
k=3	$A_3 = (s-a)^3 F(s) _{s=a}$ (cover-up)	$A_2 = d/ds[\dots] _{s=a}$; $A_1 = (1/2!) * d^2/ds^2[\dots] _{s=a}$
k umum	$A_k = (s-a)^k F(s) _{s=a}$ (cover-up)	$A_{k-j} = (1/j!) * d^j/ds^j[(s-a)^k F(s)] _{s=a}$, $j=1..k-1$

Pitfall Kasus Berulang: Pitfall paling sering pada kasus berulang adalah lupa menyertakan seluruh k term dengan pangkat berbeda; sebagian mahasiswa hanya menulis satu term $A/(s-a)^k$ tanpa term pangkat lebih rendah, padahal keduanya wajib hadir bersamaan. Pitfall kedua adalah menerapkan cover-up sederhana secara langsung untuk mendapatkan A_1 , padahal cover-up sederhana hanya valid untuk koefisien pangkat tertinggi. Verifikasi paling praktis untuk kasus berulang adalah `sp.apart()` di SymPy, yang menangani multiplicity berapapun secara otomatis dan konsisten.

KASUS 3: FAKTOR KUADRATIK DAN WORKFLOW INVERSI LENGKAP

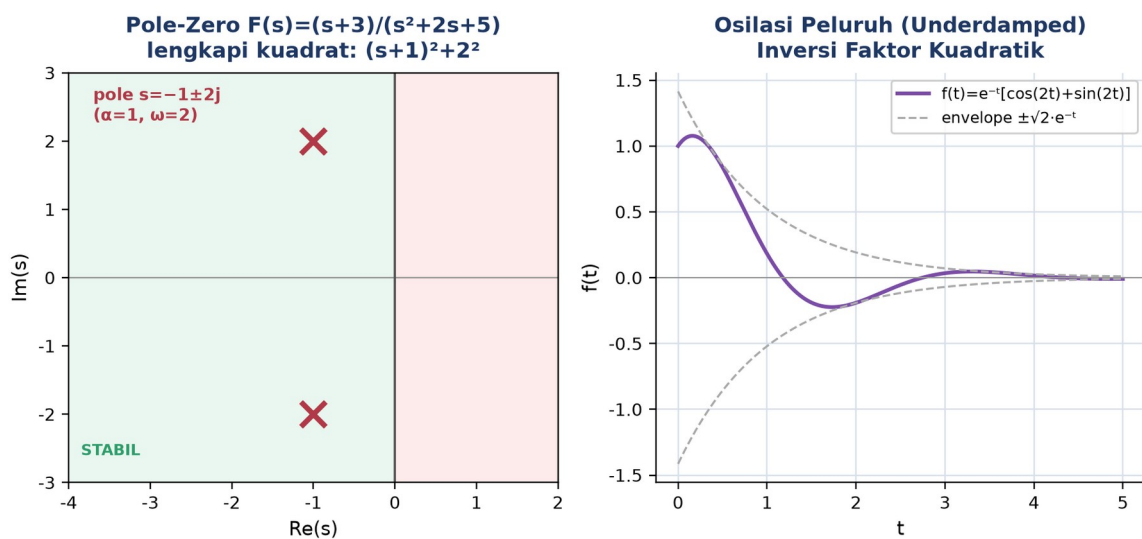
Faktor Kuadratik Tak Tereduksi. Kasus ketiga muncul bila $Q(s)$ mengandung faktor kuadratik (s^2+ps+q) dengan diskriminan $\Delta = p^2-4q$ lebih kecil dari nol, sehingga akarnya kompleks konjugat dan faktor tersebut tidak dapat direduksi lebih lanjut atas bilangan riil. Template PFD untuk kasus ini selalu memakai numerator linier $(As+B)$, bukan konstanta, karena $\deg(\text{numerator})=1$ diperlukan agar $F(s)$ tetap proper terhadap $\deg(\text{penyebut})=2$. Cover-up rule klasik tidak berlaku langsung di sini; koefisien A dan B dicari lewat menyamakan koefisien polinomial setelah mengalikan kedua ruas dengan $Q(s)$.

$P(s)/(s^2+ps+q) = (As+B)/(s^2+ps+q)$; lengkapi kuadrat: $(s+p/2)^2+(q-p^2/4) = (s+\alpha)^2+\omega^2$

Melengkapi Kuadrat dan Inversi ke Domain Waktu. Inversi kasus 3 membutuhkan satu langkah aljabar tambahan: melengkapi kuadrat pada penyebut menjadi bentuk standar $(s+\alpha)^2+\omega^2$ dengan $\alpha=p/2$ dan $\omega^2=q-p^2/4$, lalu menuliskan ulang numerator sebagai $A(s+\alpha)$ ditambah konstanta agar cocok dengan dua baris tabel: $(s+\alpha)/[(s+\alpha)^2+\omega^2]$ berkorespondensi dengan $e^{(-\alpha t)}\cos(\omega t)$, sedangkan $\omega/[(s+\alpha)^2+\omega^2]$ berkorespondensi dengan $e^{(-\alpha t)}\sin(\omega t)$. Hasil akhir selalu berupa kombinasi cosinus dan sinus dengan envelope eksponensial, signature khas sistem underdamped.

Contoh, Kuadrat dengan Damping. Tentukan PFD dan inversi Laplace dari $F(s) = (s+3)/(s^2+2s+5)$. Lengkapi kuadrat: $s^2+2s+5 = (s+1)^2+4$, sehingga $\alpha=1$ dan $\omega=2$. Tulis ulang numerator agar memuat $(s+1)$: $s+3 = (s+1)+2$. Maka $F(s) = (s+1)/[(s+1)^2+4] + 2/[(s+1)^2+4] = (s+1)/[(s+1)^2+4] + 1*2/[(s+1)^2+4]$. Suku pertama cocok pola cosinus, suku kedua (setelah faktor $2/\omega=1$) cocok pola sinus.

$$f(t) = e^{(-t)}\cos(2t) + e^{(-t)}\sin(2t) = e^{(-t)}[\cos(2t)+\sin(2t)] \quad (\text{underdamped, osilasi peluruh})$$



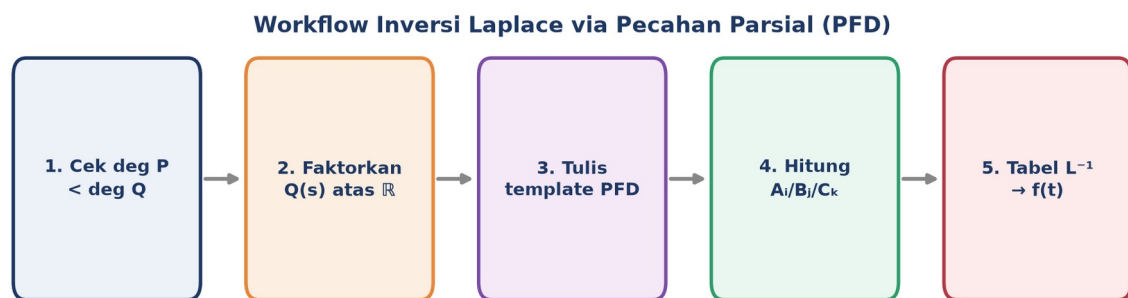
Gambar 4. Pole kompleks konjugat $s=-1 \pm 2j$ hasil melengkapi kuadrat $(s+1)^2+2^2$ (kiri) dan respons underdamped $f(t)=e^{(-t)}[\cos(2t)+\sin(2t)]$ dengan envelope peluruh (kanan).

Membaca Dua Panel di Atas: Gambar 4 menunjukkan bahwa faktor kuadrat tak tereduksi selalu berkorespondensi dengan sepasang pole kompleks konjugat di bagian kiri s-plane (bila stabil), yang di domain waktu termanifestasi sebagai osilasi dengan frekuensi ω yang meluruh mengikuti envelope eksponensial $e^{(-\alpha t)}$. Semakin besar α (bagian real pole, dengan tanda negatif), semakin cepat osilasi meluruh; semakin besar ω (bagian imajiner), semakin rapat osilasinya.

Workflow Universal Lima Langkah. Dengan tiga kasus dan cover-up rule sudah dikuasai, seluruh proses inversi Laplace via PFD dapat dirangkum menjadi satu workflow lima

langkah yang berlaku universal untuk fungsi rasional apapun, berapapun derajat penyebutnya: (1) cek syarat $\deg(P) < \deg(Q)$, lakukan long division bila perlu; (2) faktorkan $Q(s)$ atas bilangan riil menjadi kombinasi faktor linear dan kuadratik tak tereduksi; (3) tulis template PFD sesuai jenis tiap faktor; (4) hitung koefisien lewat cover-up (linear non-berulang) atau menyamakan koefisien (berulang dan kuadratik); (5) invers tiap term via tabel Laplace, lalu jumlahkan menjadi $f(t)$ total.

$F(s) \rightarrow$ cek deg $\rightarrow$ faktorkan $Q(s) \rightarrow$ template PFD $\rightarrow$ hitung $A_i/B_j/C_k \rightarrow$ tabel $L^{-1} \rightarrow f(t)$



Gambar 5. Workflow lima langkah inversi Laplace via pecahan parsial, berlaku universal untuk fungsi rasional $F(s)$ berapapun derajat penyebutnya.

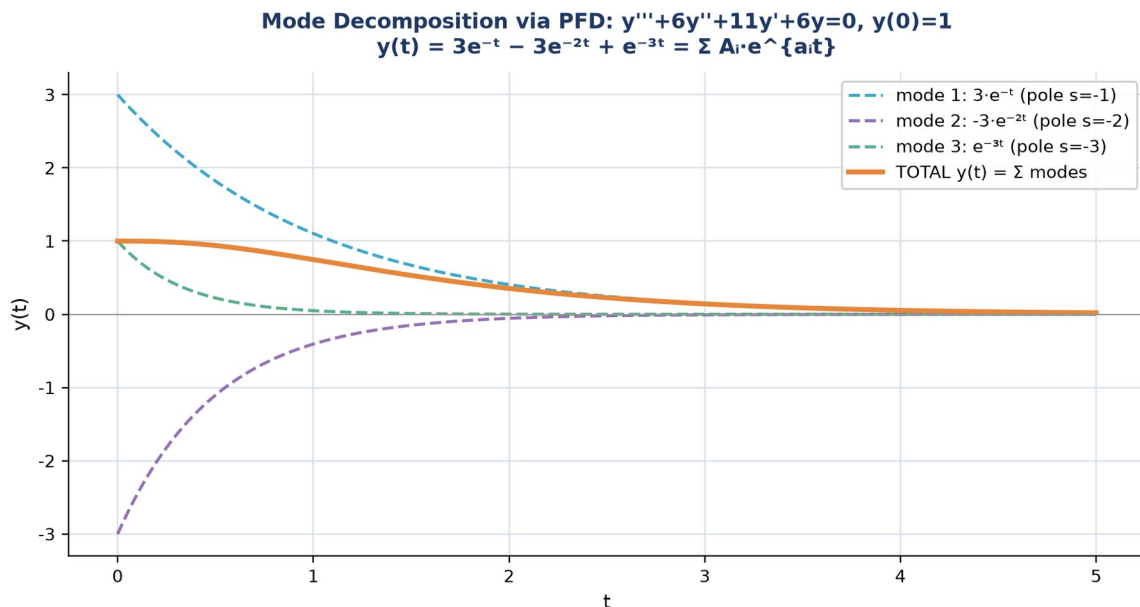
Membaca Alur Workflow: Gambar 5 merangkum kelima langkah tersebut sebagai satu alur linear yang selalu diikuti secara berurutan; kelima langkah ini identik dipakai baik untuk PD orde 2 sederhana maupun PD orde tinggi atau sistem coupled yang jauh lebih kompleks, itulah sebabnya PFD disebut sebagai algoritma deterministik, bukan trik intuisi seperti Variation of Parameters.

Contoh Terpadu, PD Orde 3 Lengkap. Selesaikan $y''' + 6y'' + 11y' + 6y = 0$ dengan $y(0)=1$, $y'(0)=0$, $y''(0)=0$ memakai workflow lengkap. Karakteristik: $s^3 + 6s^2 + 11s + 6 = (s+1)(s+2)(s+3)$, tiga akar real berbeda. Transformasi memberi $Y(s) = (s^2 + 6s + 11)/[(s+1)(s+2)(s+3)]$. Cover-up A ($s=-1$): $A=(1-6+11)/[(-1+2)(-1+3)]=6/2=3$. Cover-up B ($s=-2$): $B=(4-12+11)/[(-2+1)(-2+3)]=3/(-1)=-3$. Cover-up C ($s=-3$): $C=(9-18+11)/[(-3+1)(-3+2)]=2/2=1$.

$$\text{PFD: } Y(s) = \frac{3}{s+1} - \frac{3}{s+2} + \frac{1}{s+3} \Rightarrow y(t) = 3e^{-t} - 3e^{-2t} + e^{-3t}$$

Verifikasi Tiga Syarat Awal: Verifikasi ketiga syarat awal orde 3 memastikan hasil PFD di atas benar sebelum dipakai sebagai jawaban akhir. Turunan pertama $y'(t) = -3e^{-t} + 6e^{-2t} - 3e^{-3t}$ dan turunan kedua $y''(t) = 3e^{-t} - 12e^{-2t} + 9e^{-3t}$ diperoleh langsung dari mendiferensialkan $y(t)$ suku demi suku, sebab tiap mode eksponensial mudah diturunkan. Substitusi $t=0$ pada ketiganya memberi $y(0) = 3-3+1 = 1$ (cocok dengan syarat $y(0)=1$), $y'(0) = -3+6-3 = 0$ (cocok dengan $y'(0)=0$), dan $y''(0) = 3-12+9 = 0$

(cocok dengan $y''(0)=0$). Ketiga kecocokan ini membuktikan bahwa cover-up rule pada faktor linear berbeda tetap valid meski diterapkan pada PD orde tinggi dengan tiga syarat awal sekaligus, bukan hanya pada PD orde dua yang lebih sederhana seperti pada contoh-contoh sebelumnya.



Gambar 6. Mode decomposition solusi $y(t)=3e^{-(t)}-3e^{-(2t)}+e^{-(3t)}$: tiga mode eksponensial (garis putus-putus) bersuperposisi membentuk kurva total (garis penuh).

Mode Decomposition, Insight Struktural PFD: Gambar 6 memvisualisasikan konsekuensi struktural PFD yang sangat berguna secara fisis: setiap term hasil dekomposisi adalah satu mode natural sistem, dengan pole $s=a_i$ menentukan laju peluruhan mode tersebut dan koefisien A_i (hasil cover-up) menentukan bobot kontribusinya. Total solusi $y(t)$ tidak lain adalah superposisi ketiga mode ini, konsep yang langsung berhubungan dengan modal decomposition di control system dan structural dynamics: residu PFD identik dengan kontribusi tiap mode getar terhadap respons total struktur (Pandiya & Desmet, 2022).

Dominasi Mode dan Kaitan dengan Dominant Pole: Perhatikan bahwa ketiga pole pada contoh ini ($s=-1$, $s=-2$, $s=-3$) seluruhnya bernilai real negatif, sehingga ketiga mode meluruh menuju nol untuk t membesar tanpa osilasi sama sekali; sistem semacam ini disebut overdamped secara struktural, konsisten dengan tidak adanya faktor kuadrat tak tereduksi pada $Q(s)$ contoh ini. Mode dengan pole paling negatif ($s=-3$, mode $e^{-(3t)}$) meluruh paling cepat dan kontribusinya terhadap $y(t)$ menjadi dominan hanya pada t sangat kecil, sedangkan mode dengan pole paling dekat ke nol ($s=-1$, mode $3e^{-(t)}$) meluruh paling lambat dan pada akhirnya mendominasi bentuk kurva total untuk t yang lebih besar. Prinsip dominasi pole-terdekat-ke-sumbu-imajiner ini identik dengan konsep dominant pole di teori

kontrol, yang dipakai insinyur untuk memperkirakan perilaku transien sistem orde tinggi hanya dari satu atau dua pole paling lambat tanpa perlu menghitung seluruh mode secara eksplisit.

Tabel 3 merangkum tabel master inversi Laplace yang menggabungkan ketiga kasus PFD sekaligus, menjadi rujukan cepat saat menyelesaikan soal apapun pada Tugas Pertemuan 14 di bawah.

Tabel 3. Tabel master inversi Laplace, gabungan tiga kasus PFD (linear berbeda, linear berulang, kuadratik tak tereduksi).

$F(s)$	$f(t) = \mathcal{L}^{-1}\{F(s)\}$	Keterangan
$1/(s-a)$	e^{at}	Kasus 1: eksponensial murni
$1/(s-a)^n$	$t^{(n-1)}e^{at}/(n-1)!$	Kasus 2: polinomial x eksponensial
$\omega/(s^2+\omega^2)$	$\sin(\omega t)$	Kasus 3 ($\alpha=0$): osilasi murni
$s/(s^2+\omega^2)$	$\cos(\omega t)$	Kasus 3 ($\alpha=0$): osilasi murni
$\omega/[(s+\alpha)^2+\omega^2]$	$e^{-\alpha t}\sin(\omega t)$	Kasus 3: underdamped
$(s+\alpha)/[(s+\alpha)^2+\omega^2]$	$e^{-\alpha t}\cos(\omega t)$	Kasus 3: underdamped

ANALOGI KEHIDUPAN SEHARI-HARI

Enam analogi berikut menghubungkan konsep inti Pertemuan 14 (dekomposisi fungsi rasional, cover-up rule, tiga kasus faktor, dan mode decomposition) dengan pengalaman sehari-hari mahasiswa, mempermudah intuisi sebelum masuk ke rumus formal.

- **Membongkar Resep Masakan Campuran:** resep masakan campuran (gado-gado, rujak) sesungguhnya adalah kombinasi beberapa bahan sederhana (bumbu kacang, sayur, kerupuk) yang dicampur menjadi satu hidangan kompleks; PFD melakukan kebalikannya, memisah $F(s)$ yang rumit kembali menjadi bahan-bahan sederhana penyusunnya.
- **Menutup Satu Keran untuk Mengukur yang Lain:** menutup salah satu keran air dengan tangan untuk mengukur debit keran lain secara terpisah adalah analogi langsung cover-up rule; menutup faktor $(s-a)$ di penyebut sama seperti mengisolasi kontribusi satu pole agar dapat diukur sendiri tanpa gangguan faktor lain.
- **Paduan Suara Tiga Suara:** paduan suara dengan tiga suara berbeda (sopran, alto, bas) menghasilkan satu harmoni total; namun tiap suara tetap bisa didengar terpisah bila difokuskan, persis seperti mode decomposition PFD, tiap pole memberi satu suara (mode) yang bersuperposisi menjadi total $y(t)$.

- **Kue Lapis Legit sebagai Akar Berulang:** kue lapis legit tersusun dari lapisan-lapisan tipis identik yang ditumpuk bertahap; faktor linear berulang $(s-a)^k$ mirip struktur berlapis ini, tiap lapisan (pangkat) memberi kontribusi term $t^{(n-1)}e^{(at)}$ yang berbeda meski berasal dari akar yang sama.
- **Dua Penari Berputar sambil Melangkah Mundur:** dua penari yang berputar mengelilingi satu sama lain sambil melangkah mundur perlahan menggambarkan pole kompleks konjugat: gerakan berputar (rotasi) merepresentasikan ω (frekuensi osilasi), sementara langkah mundur yang meluruh merepresentasikan α (envelope peluruhan) pada faktor kuadrat tak tereduksi.
- **Mesin Pencari Memecah Kalimat menjadi Kata Kunci:** mesin pencari (search engine) yang memecah satu kalimat panjang menjadi kata-kata kunci terpisah untuk pencarian yang lebih efisien adalah analogi workflow lima langkah PFD; $F(s)$ yang kompleks dipecah menjadi term-term kunci sederhana agar dapat "dicari" langsung di tabel invers Laplace.

APLIKASI PFD DI INDUSTRI DAN TEKNIK MESIN

Lima Domain, Satu Teknik Dasar: Lima studi kasus berikut menunjukkan bagaimana PFD melampaui sekadar teknik aljabar kelas matematika dan menjadi standar industri di berbagai bidang rekayasa: solve PD orde tinggi, integral fungsi rasional, control system, signal processing, dan numerical analysis.

Studi Kasus 1, Solve PD Orde Tinggi via Laplace + PFD. Engineer struktur menyelesaikan PD orde tinggi hasil pemodelan sistem multi-DOF (multiple degree of freedom) lewat Laplace, menghasilkan $Y(s) = N(s)/D(s)$ dengan $\deg(D)=n$ sesuai jumlah derajat kebebasan sistem. Tanpa PFD, hasil aljabar ini mustahil di-invers kembali ke $y(t)$. Contoh: solve $y'''' + 6y''' + 11y'' + 6y' = 0$ (sistem orde 3) menghasilkan $Y(s) = (s^2 + 6s + 11)/[(s+1)(s+2)(s+3)]$; cover-up memberi $A=3, B=-3, C=1$; inversi memberi $y(t) = 3e^{-t} - 3e^{-2t} + e^{-3t}$, superposisi tiga mode natural sistem. Workflow yang sama, berapapun orde PD-nya, menjadikan PFD alat wajib analisis dinamik struktur bertingkat.

Studi Kasus 2, Integral Fungsi Rasional di Mekanika Fluida. PFD adalah teknik baku untuk menghitung integral fungsi rasional $\int P(x)/Q(x) dx$ di kalkulus lanjut dan mekanika fluida. Contoh: integral $1/[(x-1)(x+2)] dx$ memakai PFD $= (1/3)/(x-1) - (1/3)/(x+2)$; integrasi langsung memberi $(1/3)\ln|x-1| - (1/3)\ln|x+2| + C$. Aplikasi praktis meliputi perhitungan drag-flow di mekanika fluida, hambatan termal ganda pada perpindahan panas komposit multilayer, dan rate equations pada kinetika reaksi kimia; setiap kali laju alir atau

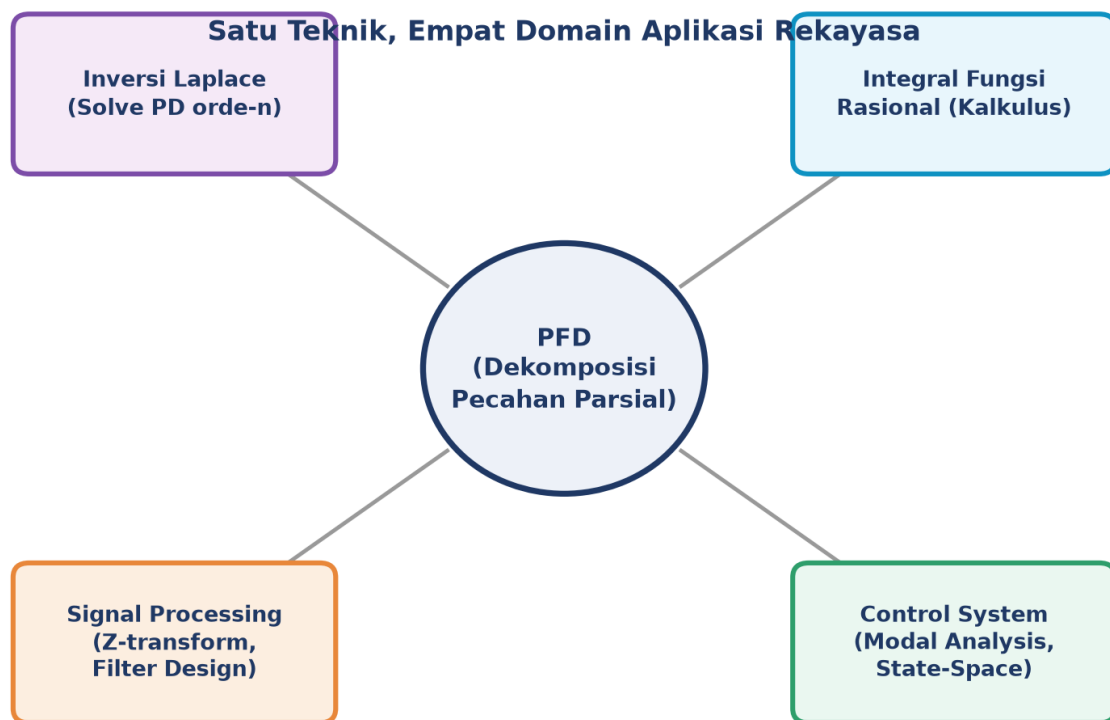
laju reaksi berbentuk fungsi rasional, PFD menjadi jalan pintas menuju solusi analitik tertutup.

Studi Kasus 3, Modal Decomposition di Control System. Di control engineering, transfer function $G(s)=Y(s)/U(s)=N(s)/D(s)$ didekomposisi PFD untuk modal decomposition: tiap pole sederhana memberi satu mode eksponensial, pole kompleks memberi mode osilasi peluruh, pole berulang memberi mode polinomial dikali eksponensial. Koefisien PFD (residu) langsung menentukan bobot kontribusi tiap mode terhadap respons total, insight yang dipakai untuk state-space realization (mengubah $G(s)$ menjadi matriks A, B, C, D) dan analisis sensitivitas pole terhadap parameter kontroler. Riset kontrol terbaru bahkan mengembangkan interpretasi baru residu PFD untuk menganalisis laju perubahan pole (pole sensitivity) terhadap parameter sistem, menghasilkan algoritma konstruksi root locus yang jauh lebih efisien secara komputasi dibanding fungsi bawaan MATLAB (Tebaldi & Zanasi, 2025).

Studi Kasus 4, Filter Digital Parallel Form via PFD. Di digital signal processing, filter IIR (infinite impulse response) direalisasikan dalam bentuk direct form atau parallel form; bentuk parallel diperoleh justru lewat partial fraction expansion pada $H(z)$, memecah transfer function orde tinggi menjadi sekumpulan filter orde satu atau dua yang dijumlahkan. Bentuk parallel ini terbukti lebih stabil secara numerik dibanding direct form, khususnya pada implementasi fixed-point dengan presisi terbatas; namun konversi naif dapat menyebabkan overflow karena magnitude respons tiap section individual bisa jauh lebih besar dari transfer function totalnya, sehingga dibutuhkan teknik delayed parallel form yang lebih hati-hati (Bank, 2018). Aplikasi nyata mencakup equalizer audio, filter noise reduction, dan matched filter di sistem komunikasi.

Studi Kasus 5, Rational Approximation di Numerical Analysis. Di numerical analysis dan pemodelan sistem, PFD dipakai untuk menyederhanakan resolvent operator linear dan rational approximation (Pade approximation) yang menggantikan fungsi transfer non-rasional atau berorde sangat tinggi dengan bentuk rasional lebih sederhana yang mudah didekomposisi. Pendekatan rational approximation berbasis kurva-fit (curve fitting) modern bahkan dipakai untuk mengubah model sistem berorde fraksional (fractional-order) menjadi transfer function rasional berorde bulat yang lebih mudah dianalisis dan diimplementasikan pada perangkat keras kontrol konvensional, sebuah teknik yang mengandalkan prinsip dekomposisi rasional yang sama persis dengan PFD (Yuce, 2023). Engineer numerik memakai pendekatan semacam ini untuk model order reduction pada sistem berdimensi besar sebelum simulasi real-time.

Benang Merah Kelima Studi Kasus: Kelima studi kasus di atas, solve PD orde tinggi, integral fungsi rasional, modal decomposition control system, filter digital parallel form, dan rational approximation numerical analysis, tampak berasal dari bidang rekayasa yang sangat berjauhan. Namun bila diperhatikan seksama, seluruhnya menyelesaikan masalah yang secara matematis identik: memecah satu ekspresi rasional kompleks menjadi jumlah komponen sederhana yang masing-masing dapat dianalisis, dihitung, atau diimplementasikan secara independen. Mahasiswa Teknik Mesin yang menguasai PFD pada mata kuliah Matematika 4 ini sesungguhnya sedang mempersiapkan diri untuk mata kuliah lanjutan seperti Sistem Kendali, Getaran Mekanik lanjut, dan Pemrosesan Sinyal, yang seluruhnya bertumpu pada kemampuan dasar dekomposisi fungsi rasional yang sama.



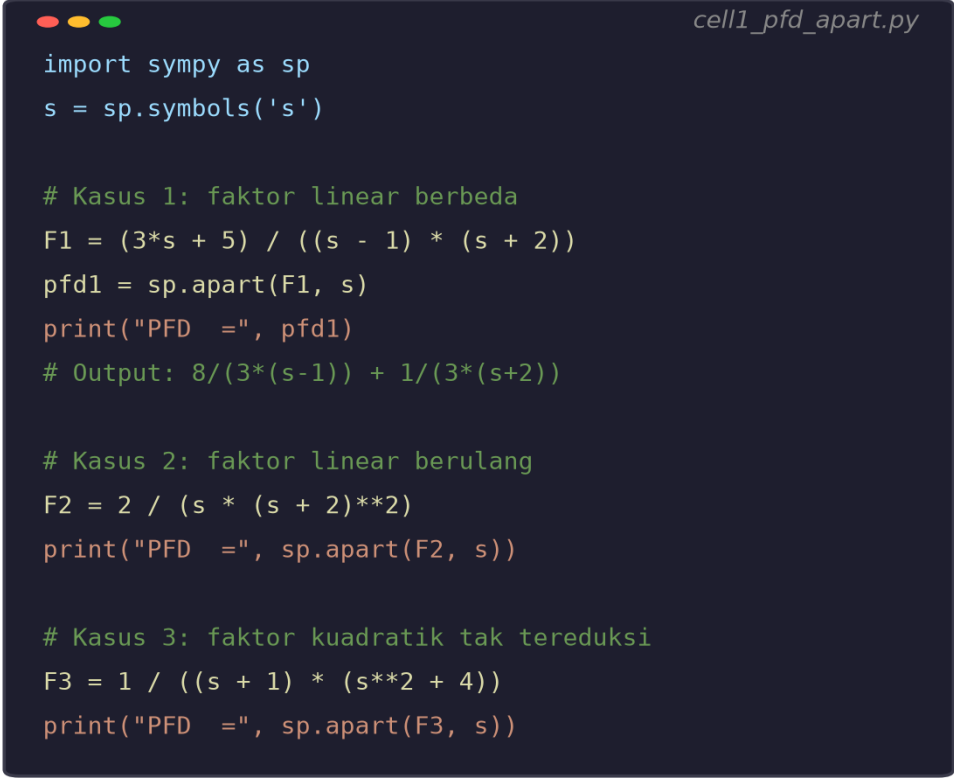
Gambar 7. Diagram hub aplikasi PFD di empat domain rekayasa: inversi Laplace untuk solve PD, integral fungsi rasional, control system (modal analysis), dan signal processing (filter design).

Satu Prinsip, Banyak Domain: Gambar 7 merangkum bahwa PFD bukan teknik tunggal; satu prinsip dasar (dekomposisi fungsi rasional menjadi jumlah suku sederhana) beriak ke berbagai domain rekayasa yang tampak tidak berhubungan pada pandangan pertama, dari solve PD hingga desain filter digital, menegaskan posisi PFD sebagai salah satu alat matematika paling serbaguna yang dipelajari mahasiswa Teknik Mesin.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Ringkasan Workflow: Python mempercepat workflow Pertemuan 14: `sp.apart()` melakukan PFD otomatis untuk ketiga kasus sekaligus, `sp.inverse_laplace_transform()` menghasilkan $f(t)$ langsung dari $F(s)$, dan implementasi manual cover-up rule (via `sp.limit()`) memastikan pemahaman algoritma di balik black-box tersebut. Empat cell berikut menunjukkan workflow lengkap dari fungsi rasional hingga solusi PD orde tinggi.

Persiapan Lingkungan: Persiapan komputasi. Pastikan environment matematika4 sudah aktif dengan paket NumPy, SciPy, SymPy, dan Matplotlib terpasang. Bila belum: (1) buka VS Code, terminal, `conda activate matematika4`; (2) `pip install numpy scipy sympy matplotlib`; (3) buka Jupyter Notebook, buat file baru, kerjakan tiap soal di tab Tugas pada cell terpisah.



```
cell1_pfd_apart.py

import sympy as sp
s = sp.symbols('s')

# Kasus 1: faktor linear berbeda
F1 = (3*s + 5) / ((s - 1) * (s + 2))
pfd1 = sp.apart(F1, s)
print("PFD =", pfd1)
# Output: 8/(3*(s-1)) + 1/(3*(s+2))

# Kasus 2: faktor linear berulang
F2 = 2 / (s * (s + 2)**2)
print("PFD =", sp.apart(F2, s))

# Kasus 3: faktor kuadrat tak tereduksi
F3 = 1 / ((s + 1) * (s**2 + 4))
print("PFD =", sp.apart(F3, s))
```

Gambar 8. Cuplikan Cell 1: PFD otomatis via `sp.apart()` untuk ketiga kasus sekaligus, faktor linear berbeda, faktor linear berulang, dan faktor kuadrat tak tereduksi.

Cell 1, PFD Otomatis dengan `sp.apart()`: Gambar 8 menunjukkan Cell 1: `sp.apart(F1, s)` pada $F1 = (3s+5)/[(s-1)(s+2)]$ otomatis memberi $8/(3*(s-1)) + 1/(3*(s+2))$ sesuai hasil manual cover-up; `sp.apart()` yang sama juga langsung menangani F2 (kasus berulang) dan F3 (kasus kuadrat) tanpa perlu mengubah kode, membuktikan bahwa satu fungsi SymPy menggeneralisasi ketiga kasus manual yang dipelajari pada modul ini.

- **Cell 1:** definisikan $s = \text{sp.symbols('s')}$; hitung sp.apart() untuk tiga $F(s)$ representatif ketiga kasus (linear berbeda, linear berulang, kuadratik tak tereduksi); verifikasi tiap hasil dengan $\text{sp.simplify}(pfd - F_asli) == 0$; tambahkan satu contoh improper rational untuk menguji long division otomatis SymPy.
- **Cell 2:** implementasikan cover-up rule manual via $\text{sp.limit}((s-a)*F, s, a)$ untuk faktor linear berbeda; verifikasi hasil sama dengan sp.apart() ; tunjukkan bahwa untuk faktor berulang, cover-up sederhana hanya memberi koefisien pangkat tertinggi (evaluasi $(s-a)^{**k} * F$ pada $s=a$), sedangkan koefisien lain harus dibaca dari sp.apart() .
- **Cell 3:** bangun workflow inversi Laplace lengkap: definisikan $Y(s)$ hasil transformasi PD (kasus underdamped, critically damped, dan tiga akar berbeda); panggil $\text{sp.inverse_laplace_transform}(Y, s, t)$ langsung; verifikasi hasil sama dengan $\text{sp.apart}(Y, s)$ diinvers term demi term secara manual via tabel.
- **Cell 4:** plot mode decomposition: pisahkan tiap term hasil PFD menjadi array NumPy terpisah (mis. $3*\text{np.exp}(-t)$, $-3*\text{np.exp}(-2*t)$, $\text{np.exp}(-3*t)$); plot ketiganya dengan garis putus-putus dan total (jumlah seluruh mode) dengan garis tebal; verifikasi $y(0)$ sesuai IVP dan $y(t)$ menuju nol untuk t besar (bila seluruh pole stabil).

Sebelum Beralih ke Tugas: Cell 1 dan Cell 3 bersifat generik: sp.apart() dan $\text{sp.inverse_laplace_transform()}$ dapat dipakai ulang untuk $F(s)$ apapun dengan mengganti definisi variabel, menjadikannya alat verifikasi cepat yang sangat berguna saat mengerjakan Tugas Pertemuan 14 di bawah. Selalu bandingkan hasil cover-up manual dengan output sp.apart() sebagai langkah terakhir sebelum menuliskan jawaban akhir, sebab kesalahan tanda pada substitusi akar negatif adalah kesalahan paling umum pada topik ini.

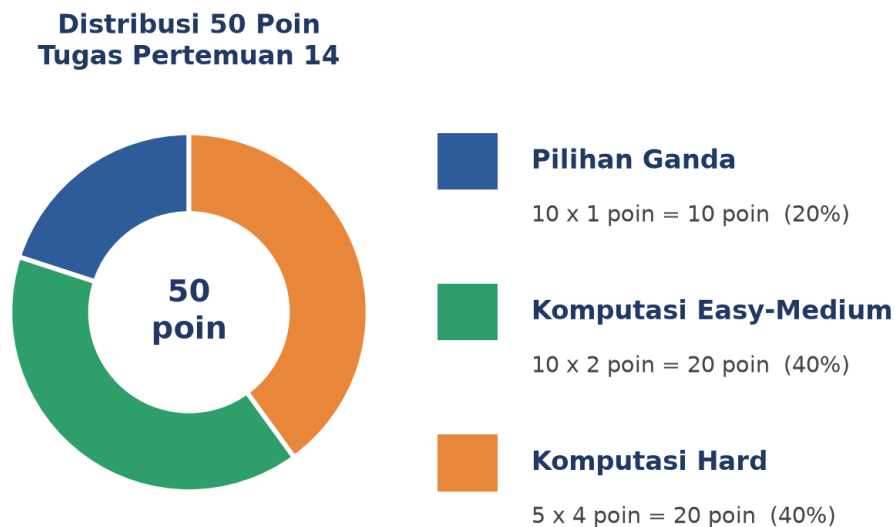
TUGAS PERTEMUAN 14

Skema Penilaian: Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Bagian A menguji pemahaman konsep tiga kasus PFD, cover-up rule, dan syarat proper secara konseptual tanpa perlu menulis kode. Bagian B menuntut evaluasi langsung dari rumus dan verifikasi numerik/symbolik sederhana yang sudah diturunkan di modul, sedangkan Bagian C menuntut implementasi algoritma lengkap dari awal (solver cover-up generik, solver faktor berulang, solver PD orde- n otomatis) sebagai uji pemahaman mendalam, bukan sekadar memanggil sp.apart() untuk bagian intinya.

Dataset Referensi Bagian B: Dataset referensi yang dipakai berulang pada Bagian B adalah fungsi rasional $F(s) = (s^2 + 6s + 11)/[(s+1)(s+2)(s+3)]$ (tiga faktor linear berbeda, dari

Contoh Terpadu PD Orde 3 di atas) dengan PFD $F(s) = 3/(s+1) - 3/(s+2) + 1/(s+3)$, serta sistem underdamped $Y(s) = (s+1)/[(s+1)^2+4] + 2/[(s+1)^2+4]$ dari Contoh Kuadratik dengan Damping. Kedua dataset ini dapat diverifikasi silang terhadap penurunan manual pada modul.

Format Pengumpulan dan Rubrik Notebook: Notebook Jupyter yang dikumpulkan wajib terstruktur rapi: satu cell markdown judul di paling atas berisi nama, NIM, dan daftar isi singkat; tiap nomor soal (Bagian B dan C) mendapat cell kode terpisah beserta satu baris komentar yang menjelaskan pendekatan sebelum kode dijalankan; dan output print() harus ditampilkan jelas (bukan hanya expression tanpa print) supaya proses grading otomatis dapat mengekstrak jawaban dengan akurat. Untuk Bagian C khususnya, sertakan minimal satu print verifikasi yang membandingkan hasil implementasi manual dengan `sp.apart()` atau perhitungan analitik dari modul; kehadiran langkah verifikasi ini menjadi salah satu kriteria penilaian partial credit bila jawaban akhir kurang tepat. Kesalahan format paling sering terjadi adalah menumpuk banyak soal dalam satu cell tanpa pemisah yang jelas, sehingga penilai kesulitan memetakan kode ke nomor soal yang dimaksud; hindari pola ini demi kelancaran proses koreksi.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 14 berdasarkan tipe soal.

Gambar 9 merangkum distribusi 50 poin di atas: Bagian A menguji pemahaman konseptual (klasifikasi faktor $Q(s)$, syarat proper, batasan cover-up rule), Bagian B menguji penerapan rumus langsung pada fungsi rasional referensi konkret, dan Bagian C menguji kemampuan mengimplementasikan algoritma PFD lengkap dari awal tanpa mengandalkan `sp.apart()` untuk bagian intinya.

Bagian A: Pilihan Ganda (10 soal x 1 poin)

1. Syarat utama agar $F(s) = P(s)/Q(s)$ dapat langsung didekomposisi PFD tanpa long division adalah...

- A. $\deg(P) > \deg(Q)$
- B. $\deg(P) < \deg(Q)$
- C. $\deg(P) = \deg(Q)$
- D. $Q(s)$ harus berderajat genap

2. Untuk $F(s)$ dengan $Q(s) = (s-1)(s+2)(s+3)$ (tiga faktor linear berbeda), template PFD yang tepat adalah...

- A. $A/(s-1)^3$
- B. $A/(s-1)+B/(s+2)+C/(s+3)$
- C. $(As+B)/[(s-1)(s+2)]$
- D. $A/s+B/(s+2)+C/(s+3)^2$

3. Cover-up rule (Heaviside method) dapat langsung diterapkan penuh untuk kasus...

- A. Faktor kuadrat tak tereduksi
- B. Faktor linear berulang, seluruh koefisien
- C. Faktor linear berbeda (non-berulang)
- D. Semua kasus tanpa kecuali

4. Untuk faktor linear berulang $(s-a)^3$, jumlah term yang wajib disertakan dalam template PFD adalah...

- A. 1
- B. 2
- C. 3
- D. Tergantung numerator

5. Template PFD untuk faktor kuadrat tak tereduksi (s^2+ps+q) selalu berbentuk...

- A. $A/(s^2+ps+q)$
- B. $(As+B)/(s^2+ps+q)$
- C. $A/(s-p)+B/(s-q)$
- D. $(As^2+B)/(s+p)$

6. Diskriminan p^2-4q lebih besar atau sama dengan nol pada faktor kuadrat (s^2+ps+q) menunjukkan bahwa faktor tersebut sebenarnya...

- A. Tetap tak tereduksi, lanjutkan kasus 3
- B. Tereduksi menjadi produk dua faktor linear
- C. Harus diabaikan
- D. Selalu memberi pole imajiner murni

7. Invers Laplace dari $1/(s-a)^2$ adalah...

- A. $e^{(at)}$
- B. $t \cdot e^{(at)}$
- C. $t^2 \cdot e^{(at)}/2$
- D. $\sin(at)$

8. Untuk faktor kuadratik yang sudah dilengkapi kuadrat menjadi $(s+\alpha)^2+\omega^2$, invers Laplace dari $\omega/[(s+\alpha)^2+\omega^2]$ adalah...

- A. $e^{(-\alpha t)} \cdot \cos(\omega t)$
- B. $e^{(-\alpha t)} \cdot \sin(\omega t)$
- C. $e^{(\alpha t)} \cdot \sin(\omega t)$
- D. $\cos(\omega t)$

9. Langkah pertama pada workflow lima langkah inversi Laplace via PFD adalah...

- A. Faktorkan $Q(s)$
- B. Cek syarat $\deg(P) < \deg(Q)$
- C. Hitung koefisien A_i
- D. Cari tabel L^{-1} langsung

10. Fungsi SymPy yang melakukan dekomposisi pecahan parsial otomatis untuk ketiga kasus sekaligus adalah...

- A. `sp.factor()`
- B. `sp.simplify()`
- C. `sp.apart()`
- D. `sp.solve()`

Bagian B: Komputasi Easy-Medium (10 soal x 2 poin)

Sistem referensi (dipakai C1-C10, definisikan sekali di Jupyter):

```
import sympy as sp
s, t = sp.symbols('s t', positive=True)
# F(s) = (s^2+6s+11)/[(s+1)(s+2)(s+3)] = 3/(s+1) - 3/(s+2) + 1/(s+3)
# Y(s) = (s+1)/[(s+1)^2+4] + 2/[(s+1)^2+4] (underdamped, alpha=1, omega=2)
```

C1. Hitung `sp.apart(F, s)` untuk $F = (s^2+6s+11)/((s+1)(s+2)(s+3))$. Cetak hasilnya dan verifikasi sama dengan $3/(s+1) - 3/(s+2) + 1/(s+3)$.

- **HINT:** Panggil `sp.apart(F, s)` langsung; bandingkan dengan `sp.simplify(hasil - (3/(s+1)-3/(s+2)+1/(s+3))) == 0`.

C2. Hitung koefisien A, B, C dari F pada soal C1 memakai cover-up rule manual: `sp.limit((s+1)*F, s, -1)`, `sp.limit((s+2)*F, s, -2)`, `sp.limit((s+3)*F, s, -3)`. Cetak ketiganya dan bandingkan dengan hasil `sp.apart()`.

- **HINT:** `sp.limit(factor*F, s, akar)` menghitung $A_i = \lim_{s \rightarrow a_i} (s - a_i) * F(s)$; ketiga hasil harus 3, -3, 1 berturut-turut.

C3. Hitung `sp.inverse_laplace_transform(F, s, t)` untuk F pada soal C1. Cetak $f(t)$ dan verifikasi $f(0)$ mendekati 1 (evaluasi numerik pada $t=0$).

- **HINT:** `sp.inverse_laplace_transform` mengembalikan ekspresi simbolik; substitusi $t=0$ lalu `sp.simplify` atau `float()` untuk verifikasi numerik.

C4. Untuk $F_2 = 2/(s*(s+2)**2)$ (faktor linear berulang), hitung `sp.apart(F2, s)`. Cetak hasilnya dan identifikasi koefisien pangkat tertinggi C pada term $C/(s+2)^2$.

- **HINT:** `sp.apart(F2, s)` memberi hasil otomatis; bandingkan term dengan penyebut $(s+2)**2$ untuk mengidentifikasi C .

C5. Verifikasi koefisien pangkat tertinggi pada soal C4 memakai cover-up dimodifikasi: `sp.limit((s+2)**2 * F2, s, -2)`. Cetak hasilnya, harus sama dengan koefisien C yang diidentifikasi pada C4.

- **HINT:** Cover-up untuk faktor berulang: kalikan F_2 dengan $(s-a)^k$ penuh (di sini $k=2$), lalu substitusi $s=a=-2$.

C6. Untuk $F_3 = 1/((s+1)*(s**2+4))$ (kasus campuran linear dan kuadrat), hitung `sp.apart(F3, s)`. Cetak hasilnya dan verifikasi jumlah pembilang $\deg=1$ pada term kuadratnya.

- **HINT:** `sp.apart(F3, s)` mengembalikan $A/(s+1) + (Bs+C)/(s**2+4)$; cetak hasil lalu identifikasi tiap koefisien dari ekspresi.

C7. Untuk $Y(s) = (s+1)/((s+1)**2+4) + 2/((s+1)**2+4)$ (dataset underdamped), hitung `sp.simplify(Y)` lalu `sp.inverse_laplace_transform(Y, s, t)`. Cetak $y(t)$ dan verifikasi $y(0)$ mendekati 1.

- **HINT:** Gabungkan kedua fraksi dengan `sp.simplify` sebelum `inverse_laplace_transform`; hasil harus $e^{-t} * [\cos(2t) + \sin(2t)]$.

C8. Hitung nilai numerik $y(t)$ pada soal C7 di $t=0,5$ dan $t=2,0$ memakai NumPy (`np.exp`, `np.cos`, `np.sin`). Cetak keduanya dan diskusikan tren (naik/turun) menuju t besar.

- **HINT:** Substitusi langsung $t=0.5$ dan $t=2.0$ ke formula $y(t)=e^{-t}*(\cos(2t)+\sin(2t))$ memakai `np.exp/np.cos/np.sin`.

C9. Untuk $F_4 = (s**3+1)/(s**2-1)$ (improper rational), hitung `sp.apart(F4, s)` dan verifikasi SymPy melakukan long division otomatis. Cetak hasilnya, harus berbentuk $s + 1/(s-1)$.

- **HINT:** `sp.apart()` otomatis mendeteksi $\deg(\text{numerator}) \geq \deg(\text{denominator})$ dan melakukan long division sebelum PFD; cetak hasil dan bandingkan bentuknya dengan $s + 1/(s-1)$.

C10. Verifikasi numerik menyeluruh: substitusi $s=5$ ke F asli pada soal C1 dan ke hasil `sp.apart()`-nya, bandingkan `float()` keduanya, cetak selisihnya (harus mendekati nol).

- **HINT:** `float(F.subs(s,5))` dan `float(pfd.subs(s,5))` harus menghasilkan angka yang identik hingga presisi numerik; cetak `abs(selisih)`.

Bagian C: Komputasi Hard (5 soal x 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal (bukan memanggil `sp.apart()` untuk bagian intinya), 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan fungsi Python `coverup_distinct(P_coeffs, roots)` yang menghitung seluruh koefisien A_i untuk kasus faktor linear berbeda TANPA memakai `sp.apart()`, memakai rumus cover-up $A_i = P(a_i) / \prod_{j \neq i} (a_i - a_j)$ secara manual (P_coeffs adalah koefisien polinomial pembilang, `roots` adalah daftar akar penyebut). Uji pada $P(s)=s^2+6s+11$, `roots=[-1,-2,-3]` (harus menghasilkan `[3,-3,1]` sesuai Contoh Terpadu PD Orde 3 di modul). Verifikasi hasil terhadap `sp.apart()` untuk $F(s)=P(s)/[(s+1)(s+2)(s+3)]$.

- **HINT:** Evaluasi $P(a_i)$ via `np.polyval` atau evaluasi manual koefisien; hitung $\prod_{j \neq i} (a_i - a_j)$ dengan loop bersarang mengecualikan indeks i ; bandingkan array hasil dengan output `sp.apart()` yang sudah dipecah term per term.

C12 (Hard). Implementasikan fungsi `repeated_root_coeffs(F_expr, s, a, k)` yang menghitung seluruh k koefisien $A_1 \dots A_k$ untuk faktor linear berulang $(s-a)^k$ memakai rumus turunan $A_{k-j} = (1/j!) \cdot d^j/ds^j [(s-a)^k F(s)]|_{s=a}$ (boleh memakai `sp.diff` dan `sp.limit`, TAPI bukan `sp.apart()` langsung untuk hasil akhir). Uji pada $F=2/(s*(s+2)**2)$, $a=-2$, $k=2$ dan verifikasi terhadap `sp.apart(F, s)`.

- **HINT:** $g = \text{sp.expand}((s-a)**k * F_expr)$; untuk $j=0$ (A_k): `sp.limit(g, s, a)`; untuk $j \geq 1$: `sp.limit(sp.diff(g, s, j), s, a) / sp.factorial(j)`; kembalikan daftar $[A_1, \dots, A_k]$ dengan urutan pangkat naik.

C13 (Hard). Implementasikan fungsi `quadratic_pfd(F_expr, s, p, q)` yang menghitung koefisien A dan B pada template $(As+B)/(s^2+ps+q)$ untuk $F(s)$ dengan SATU faktor kuadrat tak tereduksi (boleh dikombinasikan dengan faktor linear lain), memakai metode menyamakan koefisien polinomial secara manual (`sp.Poly` dan `sp.solve`, BUKAN `sp.apart()` langsung). Uji pada $F=1/((s+1)*(s**2+4))$ dan verifikasi A, B terhadap `sp.apart(F, s)`.

- **HINT:** Kalikan F dengan $Q(s)$ penuh, ekspansi `sp.expand()`, lalu `sp.Poly(...).all_coeffs()` atau samakan koefisien tiap pangkat s via `sp.solve()` terhadap sistem persamaan linier untuk A, B (dan koefisien faktor lain bila ada).

C14 (Hard). Implementasikan solver PD orde-3 generik `solve_pd_orde3_pfd(koef, ivp)` yang menerima `koef=[a3,a2,a1,a0]` dan `ivp=[y0,y0p,y0pp]`, membangun $Q(s)$ dan $G(s)$

otomatis (PD homogen, $R(s)=0$), memfaktorkan $Q(s)$ via `sp.factor_list()` atau `sp.roots()` untuk mengklasifikasikan tiap akar (linear berbeda/berulang), lalu memanggil `coverup_distinct()` dan/atau `repeated_root_coeffs()` dari soal C11/C12 (BUKAN `sp.apart()` langsung) untuk menyusun $y(t)$ simbolik. Uji pada `koef=[1,6,11,6]`, `ivp=[1,0,0]` (harus menghasilkan $y(t)=3e^{(-t)}-3e^{(-2t)}+e^{(-3t)}$ sesuai Contoh Terpadu di modul).

- **HINT:** Bangun $Q(s)=a_3s^3+a_2s^2+a_1s+a_0$ dan $G(s)$ sesuai rumus shortcut PD orde 3 dari Pertemuan 12 ($G(s)=(a_3s^2+a_2s+a_1)y_0+(a_3s+a_2)y_{0p}+a_3y_{0pp}$); $Y=G/Q$; klasifikasi akar via `sp.roots(Q,s)` (dictionary akar:multiplicity); panggil fungsi `coverup` sesuai multiplicity tiap akar.

C15 (Hard). Implementasikan fungsi `verify_pfd_numeric(F_expr, pfd_expr, s, n_points=20)` yang memverifikasi kesetaraan $F(s)$ asli dengan hasil PFD-nya secara numerik pada n_points nilai s acak (BUKAN `sp.simplify(F-pfd)==0` yang bisa lambat/gagal untuk ekspresi kompleks), mengembalikan error maksimum absolut di antara keduanya beserta boolean apakah error di bawah toleransi $1e-9$. Uji pada $F=(s^2+6s+11)/((s+1)(s+2)(s+3))$ dan `pfd=sp.apart(F,s)`, pastikan hindari titik s yang berimpit dengan pole (akar penyebut) saat generate nilai acak.

- **HINT:** Generate n_points nilai s acak (`np.random`, hindari nilai dekat akar $Q(s)$ dengan margin, mis. jarak minimum 0.1 dari tiap pole); evaluasi `float(F.subs(s,val))` dan `float(pfd.subs(s,val))` untuk tiap titik; kembalikan `np.max(np.abs(errors))` dan `errors.max() < 1e-9`.

DAFTAR PUSTAKA

- Jong, L. L., & Kuan, S. V. (2020). An effective partial fraction decomposition method for undergraduate students. *International Journal of Service Management and Sustainability*, 5(2), 163-186. <https://doi.org/10.24191/ijsms.v5i2.11717>
- Pandiya, N., & Desmet, W. (2022). Direct estimation of residues from rational-fraction polynomials as a single-step modal identification approach. *Journal of Sound and Vibration*, 517, 116530. <https://doi.org/10.1016/j.jsv.2021.116530>
- Tebaldi, D., & Zanasi, R. (2025). Applications of the partial fraction decomposition to pole variation rate for root locus construction. *IEEE Control Systems Letters*. <https://doi.org/10.1109/LCSYS.2024.3511995>
- Bank, B. (2018). Converting infinite impulse response filters to parallel form [Tips & Tricks]. *IEEE Signal Processing Magazine*, 35(3), 124-130. <https://doi.org/10.1109/MSP.2018.2805358>

Yuce, A. (2023). An approximation method for fractional-order models using quadratic systems and equilibrium optimizer. *Fractal and Fractional*, 7(6), 460.

<https://doi.org/10.3390/fractalfract7060460>